

Centres étrangers – 2025 – sujet1 - Correction

Exercice 2 (6 points)

Question 1

Réponse : 6

- *Commentaire : L'indice [1][2] correspond à la 2e ligne (indice 1) et à la 3e colonne (indice 2), donc 6.*

Question 2

```
1 def plateau_init(n, m, cartes):
2     shuffle(cartes)
3     plateau = []
4     for i in range(n) :
5         plateau.append([cartes[i+j*n] for j in range(m)])
6     return plateau
```

- *Commentaire : On utilise `shuffle` pour mélanger, puis on construit la grille ligne par ligne sans remise.*

Question 3

```
1 def cartes_voisines(n, m, i, j):
2     voisins = []
3     for i2 in range(i-1, i+2):
4         for j2 in range(j-1, j+2):
5             if (i2, j2) != (i, j) and \
6                 i2 in range(n) and \
7                 j2 in range(m):
8                 voisins.append( (i2, j2) )
9     return voisins
```

- *Commentaire : On parcourt les 8 directions autour d'une case, en vérifiant que les coordonnées sont dans la grille.*

Question 4

e1 : $9 + 8 + 6 + 7 = 30$

e2 : n'est pas une chaîne (cartes non voisines).

e3 : n'existe pas car (1,4) est hors du plateau (max colonne = 3)

- *Commentaire : Il faut additionner les valeurs aux coordonnées indiquées dans le plateau, si elles existent.*

Question 5

```
def chaine_evalue(plateau, chaine):  
    s = 0  
    for i, j in chaine:  
        s += plateau[i][j]  
    return s
```

- *Commentaire : On parcourt les coordonnées de la chaîne pour sommer les valeurs dans le plateau.*

Question 6

Le parcours en largeur explore d'abord les chemins les plus courts.

- *Commentaire : Cela permet de trouver la première solution valide avec le minimum de cartes, ce qui correspond à l'objectif du jeu.*

Question 7

```
1 def explore(plateau, cible):
2     n, m = len(plateau), len(plateau[0])
3     a_visiter = file_init()
4     for i in range(n):
5         for j in range(m):
6             file_ajoute(a_visiter, [(i,j)])
7
8     while not file_est_vide(a_visiter):
9         chemin = file_retire(a_visiter)
10        if chaine_evalue(plateau, chemin) == cible:
11            return chemin
12        dernier = chemin[-1]
13        i0, j0 = dernier
14        for i, j in cartes_voisines(n, m, i0, j0):
15            if (i, j) not in chemin:
16                file_ajoute(a_visiter,
17                           chemin + [ (i,j) ])
18
19        # Pas de solutions
20    return None
```

- *Commentaire : On utilise une file pour faire un parcours en largeur à partir de chaque case, en ajoutant des voisins valides.*

Question 8

Au lieu de retourner le premier chemin trouvé, il faut stocker tous les chemins valides dans une liste et continuer l'exploration.

On remplace `return chemin` par `chemins.append(chemin)` et on continue le parcours, puis on retourne la liste `chemins` à la fin.